

EYE ORDER COME AI

(ตาสั่งมา)



1. หัวข้อปัญหาที่ต้องการจะใช้ AI&ML ในการแก้ปัญหา

ผู้มีความบกพร่องทางร่างกายที่ไม่สามารถใช้แขน/มือได้ เช่น ผู้พิการทางแขน ผู้ป่วยอัมพาตครึ่งล่างถึงแขน หรือผู้ที่มีภาวะกล้ามเนื้ออ่อนแรง ไม่สามารถใช้มือควบคุมเมาส์และคีย์บอร์ดทั่วไปได้ อีกทั้งอุปกรณ์ Eye-tracking ระดับการแพทย์มีราคาสูง ทำให้ผู้ที่มีความบกพร่องดังกล่าวเข้าถึงได้ยาก และไม่สามารถใช้คอมพิวเตอร์ได้ ซึ่งในโลกปัจจุบัน เทคโนโลยีถือเป็นส่วนสำคัญในการดำเนินชีวิตอย่างมาก การที่กลุ่มผู้ที่มีความบกพร่องเหล่านี้เข้าถึงเทคโนโลยีไม่ได้จะทำให้พวกเขาเสียโอกาสหลายๆอย่าง และถูกทิ้งไว้ข้างหลัง

ด้วยปัญหาที่กล่าวมา จึงได้เกิดโปรเจก **“EYE ORDER COME AI”** เป็นระบบควบคุมเคอร์เซอร์เมาส์และสั่งการด้วยสายตาต้นทุนต่ำผ่านกล้องเว็บแคมทั่วไป ที่ถูกออกแบบมาเพื่อกลายขีดจำกัดด้านราคาและความยุ่งยากของอุปกรณ์แบบเดิม โดยนำเทคโนโลยี AI และ Machine Learning เข้ามาเป็นหัวใจหลักในการประมวลผลตำแหน่งการมองและถอดรหัสพฤติกรรมการกะพริบตาแบบ Real-time

เนื่องจากในความเป็นจริง ลูกตามนุษย์จะมีการสั่นไหวระดับไมโคร อยู่ตลอดเวลาตามธรรมชาติ หากใช้เพียงซอฟต์แวร์ที่คำนวณด้วยสมการเรขาคณิตพื้นฐานในการแปลงมุมมองของดวงตาเป็นพิกัดบนหน้าจอตรงๆ จะทำให้เคอร์เซอร์เมาส์สั่น กระตุก และคลิกพลาดได้ง่าย การนำ Machine Learning เข้ามาช่วยจึงช่วยเปลี่ยนสัญญาณรบกวน ปรับชดเชยความโค้งของดวงตากับระนาบหน้าจอ และทำนายตำแหน่งที่ผู้ใช้ต้องการเล็งจริงได้อย่างนิ่งและแม่นยำยิ่งขึ้น นอกจากนี้ การสั่งงานด้วยการกะพริบตายังมีความซับซ้อน เพราะมนุษย์มีการกะพริบตาตามธรรมชาติเพื่อรักษาความชุ่มชื้นควบคู่ไปกับการตั้งใจกะพริบตาเพื่อสังเกตุ ระบบจึงจำเป็นต้องพึ่งพาโมเดล AI ในการจำแนกเจตนาจากระยะเวลาและรูปแบบการเคลื่อนไหวของเปลือกตา เพื่อกรองการกะพริบตาธรรมชาติออกและตอบสนองต่อคำสั่งคลิกได้อย่างถูกต้องสิ้นไหว ซึ่งช่วยให้ผู้ที่มีความบกพร่องทางร่างกายสามารถเข้าถึงและใช้งานคอมพิวเตอร์ได้อย่างมีประสิทธิภาพ

2. แนวทางการได้มาซึ่งข้อมูลเพื่อสอนโมเดล

การพัฒนาโมเดลจำเป็นต้องใช้ข้อมูล 2 ส่วนหลัก คือ **ข้อมูลพิกัดการมอง (Regression Dataset)** สำหรับควบคุมตำแหน่งเคอร์เซอร์ และ **ข้อมูลการกะพริบตา (Classification Dataset)** สำหรับสั่งการคลิก โดยมีกระบวนการสกัดฟีเจอร์และจัดเก็บข้อมูลดังนี้

2.1) ชุดข้อมูลสำหรับการทำนายพิกัดเคอร์เซอร์เมาส์

กระบวนการสกัดฟีเจอร์นำเข้า: ใช้ MediaPipe Face Mesh 478 จุดบนใบหน้า ดึงพิกัด x, y, z เพื่อคำนวณเวกเตอร์นำเข้าทั้งหมด 7 ตัวแปร:

- Normalized Iris Position ($L_iris_ratio_x, L_iris_ratio_y, R_iris_ratio_x, R_iris_ratio_y$): คำนวณตำแหน่งจุดศูนย์กลางม่านตา เทียบกับระยะห่างสัมพัทธ์ระหว่างหัวตา และหางตา เพื่อขจัดความแตกต่างของขนาดดวงตารายบุคคล
- Head Pose Angles ($Pitch, Yaw, Roll$): คำนวณมุมองศาการหมุนของศีรษะ 3 แกน (ก้ม-เงย, หันซ้าย-ขวา, การเอียงหัวซ้าย-ขวา) ผ่านการแก้สมการ Perspective-n-Point โดยอ้างอิงพิกัดใบหน้า 6 จุดหลัก คือ ปลายจมูก, คาง, หัวตาทั้งสองข้าง และมุมปากทั้งสองข้าง เพื่อนำมาใช้ชดเชยค่าพิกัดเมื่อผู้ใช้เอียงคอหรือขยับหัว

ค่าเป้าหมายที่ต้องการทำนาย: ทำนาย “พิกัดพิกเซลจริงบนหน้าจอคอมพิวเตอร์” ($Screen_X, Screen_Y$) บนหน้าจอแสดงผล โดยระบบจะเรียนรู้ฟังก์ชันความสัมพันธ์แบบ Non-linear Mapping Function จากทั้งการเคลื่อนไหวของสายตาและองศาการหันศีรษะ ตามสมการ

$$\text{Screen Target } (Screen_X, Screen_Y) = f(\text{Gaze Ratios, Head Pose})$$

ขั้นตอนการจัดเก็บข้อมูล: ทำผ่านระบบ 9-Point Calibration ระบบแสดงหน้าต่างเต็มจอ พร้อมปุ่มแสดงจุดเป้าหมายวงกลมสีแดง ทีละ 1 จุด ครอบคลุม 9 จุดสำคัญ (มุมจอ 4 จุด, กึ่งกลางขอบจอ 4 จุด, และจุดกึ่งกลางจอ 1 จุด) โดยขณะที่ผู้ใช้จ้องมองจุดเป้าหมายแต่ละจุด ระบบจะหน่วงเวลา 0.5 วินาทีเพื่อให้สายตานิ่ง จากนั้นทำการบันทึกข้อมูลเฟรมละ 1 แถว อัตรา 30 FPS เป็นเวลา 1.5 วินาทีต่อจุด (ได้ข้อมูล 45 เฟรม/จุด รวมทั้งหมดประมาณ 405 ตัวอย่างต่อเซสชัน)

2. แนวทางการได้มาซึ่งข้อมูลเพื่อสอนโมเดล

2.2) ชุดข้อมูลสำหรับโมเดลสังการคลิก

กระบวนการสกัดฟีเจอร์นำเข้า: ดึงพิกัดเพื่อคำนวณตัวแปรวัดสัดส่วนการเปิด-ปิดตาและการเคลื่อนไหวของริมฝีปาก 3 ค่าหลัก:

- Eye Aspect Ratio - EAR (EAR_L , EAR_R): คำนวณอัตราส่วนความสูงต่อความกว้างของช่องตาจากระยะห่างของเปลือกตาบนและล่าง เทียบกับระยะหัวตา-หางตา ซึ่งค่า EAR จะมีค่าสูงเมื่อลืมตาปกติ (ประมาณ 0.25–0.35) และลดลงเข้าใกล้ศูนย์เมื่อหลับตา
- Relative Smile Ratio ($\Delta Smile$): เพื่อแก้ปัญหารูปหน้าและขนาดปากที่แตกต่างกันในแต่ละบุคคล ระบบจะบันทึกขนาดปากตอนทำหน้าปกติไว้เป็นค่าฐานในช่วงเริ่มต้น แล้วคำนวณเปอร์เซ็นต์การขยายตัวของมุมปากเทียบกับค่าฐานของตัวเอง ตามสมการ

$$\Delta Smile = \frac{Mouth_{Current} - Mouth_{Baseline}}{Mouth_{Baseline}} \times 100\%$$

ซึ่งการดูค่า EAR และ $\Delta Smile$ เพียงเฟรมเดียวไม่สามารถระบุเจตนาได้ ระบบจึงต้องเก็บค่า [EAR_L , EAR_R , $\Delta Smile$] ย้อนหลังต่อเนื่อง 30 เฟรม (เทียบเท่า 1 วินาทีที่ความเร็ว 30 FPS) ในลักษณะคิวเลื่อน (Sliding Queue) ทำให้โมเดลเห็นลำดับเหตุการณ์ ความเร็ว และระยะเวลาในการปิดตา เพื่อแยกแยะคำสั่งได้อย่างแม่นยำ

2. แนวทางการได้มาซึ่งข้อมูลเพื่อสอนโมเดล

2.2) ชุดข้อมูลสำหรับโมเดลสั่งการคลิก (ต่อ)

กลุ่มข้อมูลและคลาสเป้าหมาย: บันทึกตัวอย่างรูปแบบพฤติกรรมใบหน้าจากผู้ใช้งาน แบ่งเป็น 5 คลาสหลัก (เก็บตัวอย่างรูปแบบละ 100–150 ชุด ข้อมูล)

- Class 0 - Normal Blink (กะพริบตาธรรมชาติ): ค่า EAR ลดลงและดีดกลับมาลึบตามปกติอย่างรวดเร็วภายใน 100 - 200 ms (3–6 เฟรม) → ระบบเพิกเฉย ไม่สั่งคลิก
- Class 1 - Left Wink (คลิกซ้าย): ค่า EAR_L ลดต่ำลงค้างไว้ 350 - 600 ms (10–18 เฟรม) ขณะที่ตาขวาลึบตามปกติ → สั่ง คลิกซ้าย
- Class 2 - Right Wink (คลิกขวา): ค่า EAR_R ลดต่ำลงค้างไว้ 350 - 600 ms (10–18 เฟรม) ขณะที่ตาซ้ายลึบตามปกติ → สั่ง คลิกขวา
- Class 3 - Double Blink (ดับเบิลคลิก): ค่า EAR ลดลงและดีดขึ้น 2 รอบติดต่อกันภายในกรอบเวลาไม่เกิน 500 ms} (ไม่เกิน 15 เฟรม) → สั่ง Double Click
- Class 4 - Extended Wink / Drag Mode (คลิกซ้าย/ขวาค้าง): ค่า EAR_L หรือ EAR_R ลดต่ำลงค้างไว้นานเกิน 1,000 ms (> 30 เฟรม) → สั่ง Mouse Down ค้างไว้เพื่อลากวัตถุหรือคลุมข้อความ และจะสั่ง Mouse Up เมื่อตรวจพบการกะพริบตาซ้ำอีก 1 ครั้ง
- Class 5 - Smile & Head Nod Gesture (โหมดยิ้ม/พย�หน้าจ่อ / Scroll): ค่า $\Delta Smile +25\%$ ต่อเนื่องเกิน 200 ms → ล็อกตำแหน่งเคอร์เซอร์ แล้วส่งคำสั่ง Scroll Up/Down ตามองศาหัว-เงยของศีรษะ

ขั้นตอนการจัดเก็บข้อมูล: ทำผ่านระบบ Prompt-Guided Gesture Collection โดยระบบจะแสดงหน้าต่างแนะนำท่าทางพร้อมภาพตัวอย่างและแถบเวลานับถอยหลังทีละคำสั่ง เริ่มจากการให้ผู้ใช้นั่งมองจอด้วยสีหน้าปกติ 3 วินาทีเพื่อบันทึกค่าฐาน จากนั้นระบบจะสุ่มแสดงคำสั่งท่าทาง (เช่น กะพริบตาธรรมชาติ, ขยับตาซ้าย/ขวา, กะพริบตาสองครั้ง, ขยับตาค้างยาว, และยิ้มยั้งฟัน) พร้อมให้สัญญาณเริ่มท่าทาง โดยระบบจะบันทึกข้อมูลค่าสัดส่วนใบหน้า [EAR_L , EAR_R , $\Delta Smile$] แบบอนุกรมเวลาในหน้าต่าง 30 เฟรม (1 วินาที) ที่อัตรา 30 FPS พร้อมกำกับป้ายคลาส ให้อัตโนมัติ ทำละ 20–30 รอบต่อคน (รวมประมาณ 100–150 ตัวอย่างต่อคลาส)

3. เครื่องมือและไลบรารีที่คาดว่าจะใช้

ภาษาหลักในการพัฒนา

- Python

Computer Vision & Landmark Extraction

- OpenCV: จัดการสตรีมภาพจากกล้องเว็บแคมและการประมวลผลภาพพื้นฐาน
- MediaPipe Face Mesh: สกัดพิกัด 3 มิติ 478 จุดบนใบหน้า และสกัดจุดศูนย์กลางม่านตา

Machine Learning & Data Processing

- Scikit-learn: สำหรับการเทรน ปรับจูนไฮเปอร์พารามิเตอร์ และประเมินโมเดล Classification และ Regression
- NumPy และ Pandas: จัดการโครงสร้างข้อมูลเวกเตอร์ อาร์เรย์ และการทำ Sliding Window

OS & Hardware Controller

- PyAutoGUI: ส่งคำสั่งควบคุมระบบปฏิบัติการ (เลื่อนเคอร์เซอร์, คลิกซ้าย, คลิกขวา, ดับเบิลคลิก, เลื่อนหน้าจอ)

User Interface

- PyQt6 หรือ CustomTkinter: ออกแบบหน้าต่าง Calibration, เมนูตั้งค่า และ Overlay แสดงสถานะ

4. อัลกอริทึม ML ที่คาดว่าจะใช้และเปรียบเทียบ

โมเดลสำหรับทำนายพิกัดเมาส์

อัลกอริทึม	ข้อดี	ข้อเสีย / ข้อจำกัด
Ridge / Polynomial Regression	ประมวลผลเร็วระดับ Microseconds ใช้ทรัพยากรต่ำมาก ไม่หน่วงระบบ	จัดการความสัมพันธ์ไม่เป็นเส้นตรง (Non-linear) ที่ซับซ้อนได้จำกัดเมื่อมุมศีรษะเอียงมาก
Support Vector Regression (SVR - RBF Kernel)	แม่นยำสูงกับข้อมูลมิติน้อย จัดการ Non-linear ได้ดีมาก ทนต่อ Outlier	ใช้เวลาเทรนและประมวลผลมากกว่าเชิงเส้นเล็กน้อย ต้องจูนค่า C และ γ อย่างละเอียด
Random Forest Regressor	จับความสัมพันธ์ซับซ้อนระหว่างตากับมุมหัวได้ดี ไม่เกิด Overfitting ง่าย	ขนาดโมเดลใหญ่กว่า และอาจเกิดอาการพิกัดเมาส์กระโดดเป็นขั้นบันได (Discontinuous Output)

4. อัลกอริทึม ML ที่คาดว่าจะใช้และเปรียบเทียบ

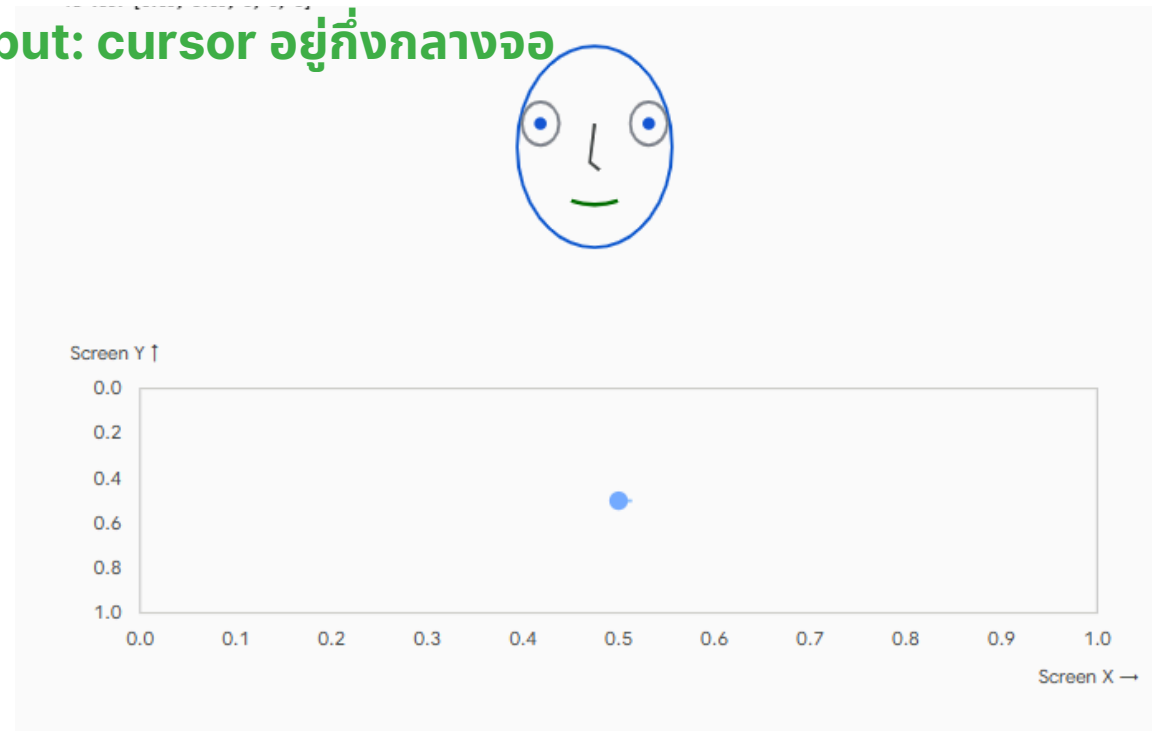
โมเดลสำหรับจำแนกคำสั่งคลิก

อัลกอริทึม	ข้อดี	ข้อเสีย / ข้อจำกัด
Support Vector Machine (SVM - RBF)	แยกคลาสพฤติกรรมเวลาได้แม่นยำสูงเมื่อป้อน Sliding Window ขนาด 30×3	ต้องการข้อมูลที่ผ่านการ Normalize สเกลมาอย่างดี
Random Forest Classifier	ทนทานต่อสัญญาณรบกวน (Noise) แยกแยะความเร็วของการกะพริบตาได้ชัดเจน	ใช้หน่วยความจำมากกว่าเชิงเส้น ในการเก็บต้นไม้ตัดสินใจหลายต้น
1D-CNN (1D Convolutional Neural Network)	สกัดคุณลักษณะของสัญญาณอนุกรมเวลา (Time-series) ได้อัตโนมัติ แม่นยำสูงสุด	ต้องใช้ชุดข้อมูลฝึกสอนจำนวนมาก และกินพลังงานประมวลผลสูงกว่าอัลกอริทึมข้างต้น

5. ตัวอย่างผลลัพธ์ (MOCKUP)

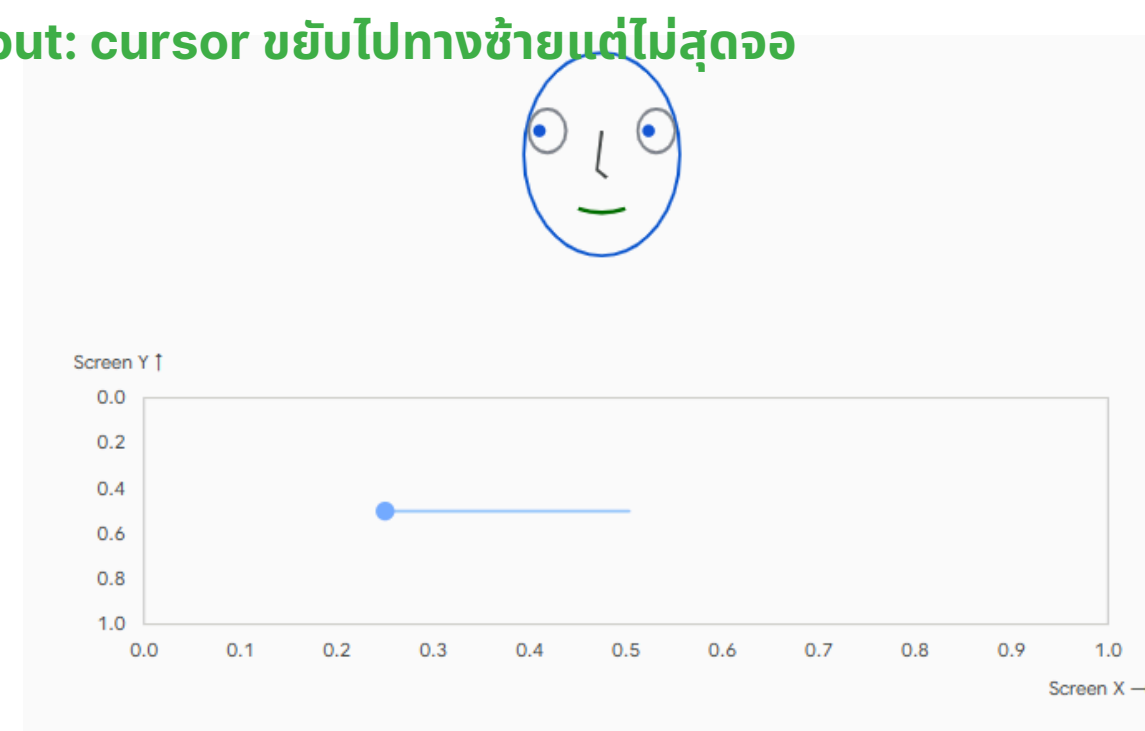
input: ตามองตรง+หน้าตรง

output: cursor อยู่กึ่งกลางจอ



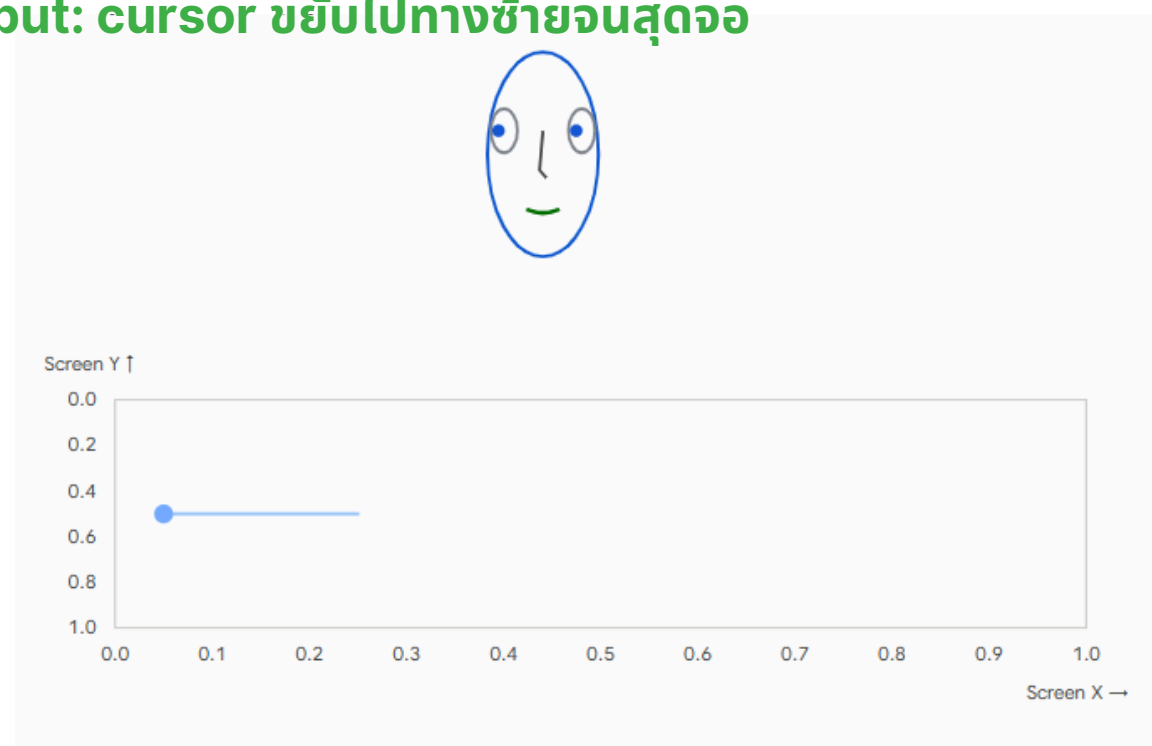
input: ตามองทางซ้าย+หน้าตรง

output: cursor ขยับไปทางซ้ายแต่ไม่สุดจอ



input: ตามองซ้าย+หน้าหันทางซ้าย

output: cursor ขยับไปทางซ้ายจนสุดจอ



input: ตามองซ้าย+หน้าหันทางซ้าย+ก้มหน้า

output: cursor ขยับไปทางมุมล่างซ้ายจนสุดจอ

